تعريف الكلاس والكائنات

[القاعدة الذهبية #1]: الكلاس نوع بيانات **مُعرَّف من قِبَل المبرمج** يحتوي على Data Members + Member Functions، والاختلاف الجوهري عن `struct` في C هو **إخفاء البيانات (Data Hiding)**

[القاعدة الذهبية #2]: العضو بدون تحديد محدد وصول هو **private افتراضياً** — هذه من أكثر الأسئلة تكراراً

```cpp
class item
{
    int number;     // private افتراضياً بدون كتابة private
    float cost;     // private افتراضياً
public:
    void getdata(int a, float b);
    void putdata(void);
};
```

[جدول المقارنة — محددات الوصول]:

المحدد	الوصول من داخل الكلاس	الوصول من خارج الكلاس	الافتراضي؟
`private`	✅ نعم	❌ لا	✅ **نعم**
`public`	✅ نعم	✅ نعم	❌ لا
`protected`	✅ نعم	❌ لا (إلا المشتقة)	❌ لا

[سؤال امتحاني متوقع #1]:

> في الكلاس التالي، ما محدد الوصول لـ `x` و `y`؟
>
> ```cpp
> class xyz { int x; int y; public: int z; };
> ```
>
> - أ) x وy عامان، z خاص
> - ب) ✅ x وy خاصان افتراضياً، z عام
> - ج) الكل عام
> - د) الكل خاص

إنشاء الكائنات والوصول للأعضاء

[القاعدة الذهبية #3]: صيغة الوصول للدوال العامة عبر الكائن هي:

```text
object_name . function_name(arguments);
```

[منطقة الخطر #1 — الفخ الأكثر شيوعاً]:

```cpp
class xyz { int x; int y; public: int z; };
xyz p;

p.x = 0;   // ❌ ERROR — x خاص، لا يُصَل إليه مباشرة
p.z = 10;  // ✅ OK   — z عام [file:71]
```

[سؤال امتحاني متوقع #2]:

> ما نتيجة تنفيذ `x.number = 100` إذا كان `number` عضواً خاصاً في الكلاس؟
>
> - أ) يعمل بشكل صحيح
> - ب) Runtime Error
> - ج) ✅ Compile-time Error
> - د) Warning فقط

تعريف دوال الأعضاء

[جدول المقارنة — Inside vs Outside Class]:

العنصر	تعريف داخل الكلاس	تعريف خارج الكلاس
الصيغة	مباشرة داخل `{ }`	`()ReturnType ClassName::func`
المعامل `::`	لا يحتاجه	**إلزامي**
سلوك Inline	يُعامَل كـ inline تلقائياً	دالة عادية
الاستخدام المفضل	دوال قصيرة جداً	دوال طويلة أو معقدة

[**القاعدة الذهبية #4**]: الرمز `::` يُسمى **Scope Resolution Operator** ويُخبر المُصرّف بأن هذه الدالة تنتمي لكلاس معين

```cpp
// تعريف خارج الكلاس — الصيغة الصحيحة
void item :: getdata(int a, float b)
{
    number = a;    // ✅ وصول مباشر لـ private من داخل الكلاس
    cost = b;
}
```

[**القاعدة الذهبية #5**]: دالة العضو يمكنها استدعاء دالة عضو أخرى **مباشرةً بدون نقطة** — لأنها داخل نفس الكلاس

تداخل الدوال (Nesting) والدوال الخاصة

[القاعدة الذهبية #6 — Nesting]:

دالة عضو يمكنها استدعاء دالة عضو أخرى في **نفس الكلاس** مباشرةً بدون dot operator

```cpp
void set::display(void)
{
    cout << largest();  // ✅ استدعاء مباشر بدون نقطة
}
```

[منطقة الخطر #2 — Private Member Function]:

حتى **الكائن نفسه** لا يمكنه استدعاء دالة خاصة من الخارج — فقط دوال الأعضاء داخل الكلاس هي من تستطيع ذلك

```cpp
class sample { void read(void); };  // read خاصة
sample s;
s.read();  // ❌ ILLEGAL — حتى الكائن لا يملك صلاحية!
```

الأعضاء الساكنة(Static)

[القاعدة الذهبية #7 — Static Data Member]:

ثلاث خصائص لا تتغير أبداً:

- **نسخة واحدة فقط** مشتركة بين جميع الكائنات
- تُهيَّأ بـ **صفر تلقائياً** عند إنشاء أول كائن
- **يجب تعريفها خارج الكلاس** بصيغة: `int ClassName::member;`

```cpp
class item {
    static int count;  // إعلان داخل الكلاس
};
int item::count;  // ✅ تعريف إلزامي خارج الكلاس [file:71]
```

[القاعدة الذهبية #8 — Static Member Function]:

الدالة الساكنة يمكن استدعاؤها **بدون كائن** عبر اسم الكلاس مباشرة، لكنها **لا تصل** إلا للأعضاء الساكنة فقط

```cpp
test::showcount();  // ✅ استدعاء بدون كائن
```

[جدول المقارنة — Static Member vs Normal Member]:

الخاصية	عضو عادي	عضو ساكن
عدد النسخ	نسخة لكل كائن	**نسخة واحدة للكل**
الاستدعاء	يحتاج كائن	`ClassName::func()`
الوصول	كل الأعضاء	**الأعضاء الساكنة فقط**
التهيئة	عند إنشاء الكائن	**صفر تلقائياً**

[تمرين #1 Tracing] — What is the output?

```cpp
class item {
    static int count;
    int number;
public:
    void getdata(int a) { number=a; count++; }
    void getcount() { cout << count << endl; }
};
int item::count;

int main() {
    item a, b, c;
    a.getcount();   // سطر 1
    a.getdata(10);
    b.getdata(20);
    c.getdata(30);
    a.getcount();   // سطر 2
    b.getcount();   // سطر 3
}
```

السطر	المخرج	السبب
سطر 1	0	لم يُستدعَ `getdata` بعد
سطر 2	3	count مشترك — زيد 3 مرات
سطر 3	3	**نفس النسخة** لكل الكائنات

الكائن كوسيط(Objects as Arguments)

[جدول المقارنة — Pass by Value vs Pass by Reference للكائنات]:

المعيار	Pass by Value	Pass by Reference
ما يُمرَّر	**نسخة كاملة** من الكائن	**عنوان** الكائن فقط
التأثير على الأصل	❌ لا يؤثر	✅ يؤثر مباشرة
الكفاءة	أبطأ (نسخ كامل)	**أسرع** (عنوان فقط)

الـ Constructor (البنّاء)

[القاعدة الذهبية #9 — خصائص Constructor الست]:

- ✅ يجب أن يكون في `public`
- ✅ يُستدعى تلقائياً عند إنشاء الكائن
- ❌ لا يملك نوع إرجاع — ولا حتى `void`
- ❌ لا يُورَّث (لكن الكلاس المشتق يمكنه استدعاؤه)
- ✅ يمكنه امتلاك Default Arguments
- ❌ لا يمكن أن يكون `virtual`

[جدول المقارنة — أنواع الـ Constructor]:

النوع	الصيغة	متى يُستدعى؟
Default	`()ClassName`	`;ClassName obj`
Parameterized	`ClassName(int x, int y)`	`;ClassName obj(1, 2)`
Copy	`ClassName(ClassName &obj)`	`;ClassName B(A)` أو `B=A`
Dynamic	يستخدم `new` داخله	عند الحاجة لتخصيص ذاكرة

[منطقة الخطر #3 — Parameterized Constructor]:

طريقتان للاستدعاء، كلتاهما صحيحتان:

```cpp
integer int1(0, 100);           // ✅ Implicit call (الأكثر شيوعاً)
integer int1 = integer(0, 100); // ✅ Explicit call
```

[تمرين #2 Tracing] — What is the output?

```cpp
class integer {
    int m, n;
public:
    integer(int x, int y) { m=x; n=y; }
    void display() { cout<<"m="<<m<<" n="<<n<<endl; }
};
int main() {
    integer int1(0, 100);
    integer int2 = integer(25, 75);
    int1.display();
    int2.display();
}
```

الكائن	المخرج
int1	✅ m=0 n=100
int2	✅ m=25 n=75

Destructor (المدمّر)

[القاعدة الذهبية #10 — خصائص Destructor الأربع]:

- اسمه = اسم الكلاس مسبوقاً بـ `~`
- **لا يأخذ وسائط** ولا يُرجع قيمة
- يُستدعى **تلقائياً** عند الخروج من نطاق الكائن
- **يُحرّر الذاكرة** — مهم جداً مع `new`

```cpp
~integer() { }          // Destructor بدون وسائط
```

[جدول المقارنة — Constructor vs Destructor]:

الخاصية	Constructor	Destructor
الاسم	= اسم الكلاس	`~` + اسم الكلاس
وسائط	✅ مسموح	❌ ممنوع
نوع الإرجاع	❌ لا يوجد	❌ لا يوجد
وقت الاستدعاء	عند **إنشاء** الكائن	عند **تدمير** الكائن
التحميل الزائد	✅ مسموح	❌ ممنوع
`virtual`	❌ ممنوع	✅ مسموح

🏆 أسئلة MCQ الأكثر توقعاً

[سؤال #3]:

> ما الصحيح بخصوص الـ Constructor؟
>
> - أ) يمكن أن يكون `virtual`
> - ب) ✅ لا يملك نوع إرجاع ولا حتى `void`
> - ج) يُورَّث للكلاسات المشتقة
> - د) يُستدعى يدوياً فقط
>
> 📋

[سؤال #4]:

> أيٌّ من العبارات التالية صحيحة بخصوص الـ Static Data Member؟
>
> - أ) كل كائن يملك نسخته الخاصة
> - ب) تُهيَّأ بـ 1 عند إنشاء أول كائن
> - ج) ✅ نسخة واحدة مشتركة بين جميع الكائنات وتُهيَّأ بصفر
> - د) لا يمكن تعريفها خارج الكلاس
>
> 📋

[سؤال #5 — Copy Constructor]:

> أيٌّ من الجمل التالية لا تستدعي الـ Copy Constructor؟
>
> - أ) `code B(A);`
> - ب) `code C = A;`
> - ج) ✅ `D = A;` (بعد إنشاء D مسبقاً — هذا assignment وليس copy constructor)
> - د) `code E(A);`
>
> 📋